asymptotically equivalently expensive to compute the single-source-single-destination shortest path, and the single-source-all-vertices shortest paths[6].

If we were interested in all the distances between all the pairs of vertices, instead, we would have better options than running BFS |V| times (but that's out of scope here).

14.3.3 Reconstructing the Path to Target

Quite often, besides computing the minimum distance of a certain vertex from a source vertex, we are also interested in discovering what's the shortest path to that vertex, meaning which edges should be followed, and in which sequence, to get from source to destination.

As shown in the bottom part of figure 14.10, we can obtain a tree considering the "parent" relation between visited and discovered vertices: each vertex u has exactly one parent, the vertex v that was being visited when u was discovered (line #13 in listing 14.2).

Listing 14.3 The path reconstruction method

```
function reconstructPath(parents,
destination)                                    #1
  if parents[destination] == null
then                                            #2
    return null
  current ← destination
  path ←
[destination]
```